

BYLON Malick Ciel2

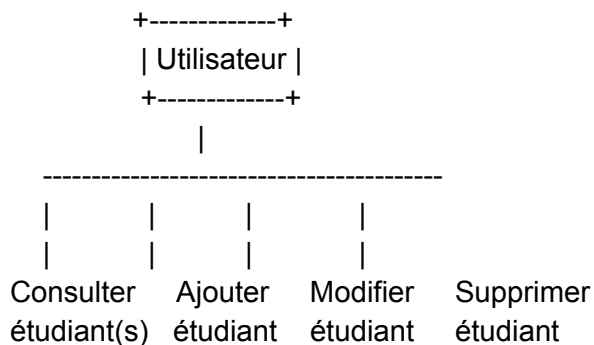
1. Présentation du projet

L'objectif de ce projet était de développer une API REST en Python permettant de gérer une liste d'étudiants enregistrés dans une base de données MySQL nommée **ciel2027**. L'API utilise le framework Flask pour recevoir les requêtes HTTP et la bibliothèque mysql.connector pour communiquer avec la base de données. La connexion à la base est réalisée avec l'utilisateur root sur l'adresse 127.0.0.1.

Les fonctionnalités principales de l'API sont :

- Consulter tous les étudiants
- Consulter un étudiant précis
- Ajouter un étudiant
- Modifier un étudiant
- Supprimer un étudiant

Diagramme de cas d'utilisation



2. Description de l'API

L'API est basée sur le protocole HTTP et renvoie les résultats au format JSON.

GET /v2/etudiants/

Permet de récupérer la liste complète des étudiants enregistrés dans la base de données. La fonction exécute une requête SQL SELECT puis transforme chaque ligne en objet JSON.

Réponse

```
[
  {
    "idetudiant": 1,
    "nom": "Dupont",
    "prenom": "Jean",
    "email": "jean@mail.fr",
    "telephone": "0696000000"
  }
]
```

Code HTTP :

200 OK

GET /v2/etudiants/{id}

Permet de récupérer les informations d'un étudiant à partir de son identifiant. La requête SQL utilise une f-string pour filtrer l'identifiant demandé.

Réponses

200 OK

404 Not Found

POST /v2/etudiants/

Permet d'ajouter un nouvel étudiant dans la base de données. Les informations sont transmises dans le corps de la requête au format JSON.

Exemple JSON

```
{
  "nom": "BYLON",
  "prenom": "Malick",
  "email": "malick@mail.fr",
  "telephone": "0696123456"
}
```

Réponse

201 Created

PUT /v2/etudiants/{id}

Permet de modifier les informations d'un étudiant existant. La requête UPDATE est exécutée sur l'identifiant fourni dans l'URL.

Réponses

200 OK
404 Not Found

DELETE /v2/etudiants/{id}

Permet de supprimer un étudiant de la base de données.

Réponses

200 OK
404 Not Found

3. Historique des versions

Version 1

La première version de l'API implémente l'ensemble des opérations CRUD :

- GET tous les étudiants
- GET un étudiant
- POST ajout
- PUT modification
- DELETE suppression

Les routes utilisent le préfixe **/v1/**.

Exemple

/v1/etudiants/

Version 2

La version 2 ajoute la gestion des erreurs avec les blocs try/except.

Améliorations :

- Gestion des erreurs 404 lorsque l'identifiant n'existe pas.
 - Gestion des erreurs serveur 500.
 - Contrôle des données reçues lors des ajouts.
 - Vérification du nombre de lignes modifiées ou supprimées avec rowcount.
-

Version 3

Fonctionnalités prévues :

- Création d'une table user.
- Authentification utilisateur.
- Route POST /login.
- Création d'une classe Database dans un fichier db.py.
- Programmation orientée objet.

Ces fonctionnalités n'ont pas encore été implémentées dans les fichiers fournis.

4. Diagramme de classes

Classe Database prévue pour la version 3 :

```
+-----+
|   Database   |
+-----+
| host         |
| user         |
| password     |
| database     |
+-----+
| connect()    |
| getEtudiants() |
| getEtudiant() |
| createEtudiant() |
| updateEtudiant() |
| deleteEtudiant() |
| login()      |
+-----+
```

Diagramme généré avec :

pyreverse -o png db.py

5. Tests réalisés avec Postman

Test	URL	Résultat
GET tous les étudiants	/v2/etudiants/	OK
GET étudiant existant	/v2/etudiants/1	OK
GET étudiant inexistant	/v2/etudiants/999	404
POST ajout étudiant	/v2/etudiants/	201
PUT modification étudiant	/v2/etudiants/1	200
DELETE étudiant	/v2/etudiants/1	200

6. Conclusion

Ce projet a permis de développer une API REST complète en Python à l'aide du framework Flask et d'une base de données MySQL. Les opérations CRUD ont été mises en place avec succès et les données sont échangées au format JSON.